

Tecnologias de Comunicação para Sistemas Críticos

Laboratory Class PL3 2026/27 Edition – 3 hours

Arduino set up & UDP & TCP protocols

Contents

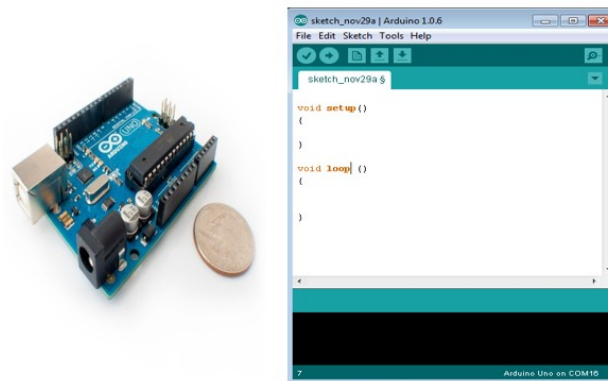
1. Arduino Uno Board.....	2
ESP32-S3 pinout:.....	6
2. Arduino IDE.....	7
2.1 Installation.....	7
2.2.1 Windows only.....	9
2.2.2 Linux only useful commands.....	11
2.2 Memory Management.....	11
2.2.1 Arduino IDE Memory Configuration.....	13
3. [Lab03.ex01] Internal LED blink.....	14
3.1 ESP32S3 [Lab03.ex01] Internal LED blink.....	15
4. [Lab03.ex02] Hello World.....	17
5. [Lab03.ex03] LED On.....	17
6. [Lab03.ex04] Arduino Client – SRV01 Server – UDP connection.....	17
7. [Lab03.ex05] Error handling.....	21
8. [Lab03.ex06] Lights On.....	21
9. [Lab03.ex07] Arduino TCP Server – Client connection.....	21
10. [Lab03.ex08] Arduino TCP Server – Light On.....	23

Arduino is a prototype platform (open-source) based on an easy-to-use hardware and software. It consists of a circuit board, which can be programmed (referred to as a microcontroller) and a ready-made software called Arduino IDE (Integrated Development Environment), which is used to write and upload the computer code to the physical board.

The key features are:

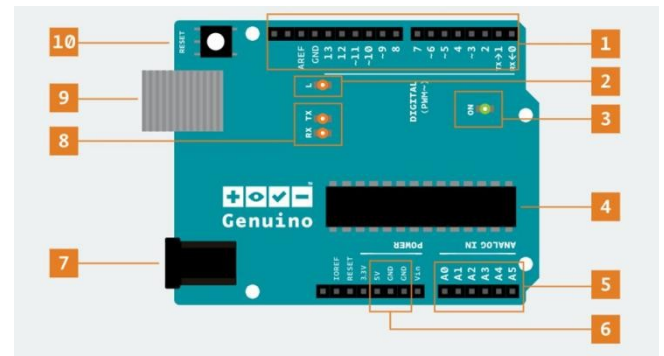
- Arduino boards are **able to read analogue or digital input signals** from different sensors and turn it **into an output such as activating a motor, turning LED on/off, connect to the cloud and** many other actions.
- You can control your board functions by sending a set of instructions to the microcontroller on the board via Arduino IDE (referred to as uploading software).
- Unlike most previous programmable circuit boards, Arduino does not need an extra piece of hardware (called a programmer) in order to load a new code onto the board. You can simply use a USB cable.

- Additionally, the Arduino IDE **uses a simplified version of C++**, making it easier to learn to program.
- Finally, Arduino provides a standard form factor that breaks the functions of the micro-controller into a more accessible package.

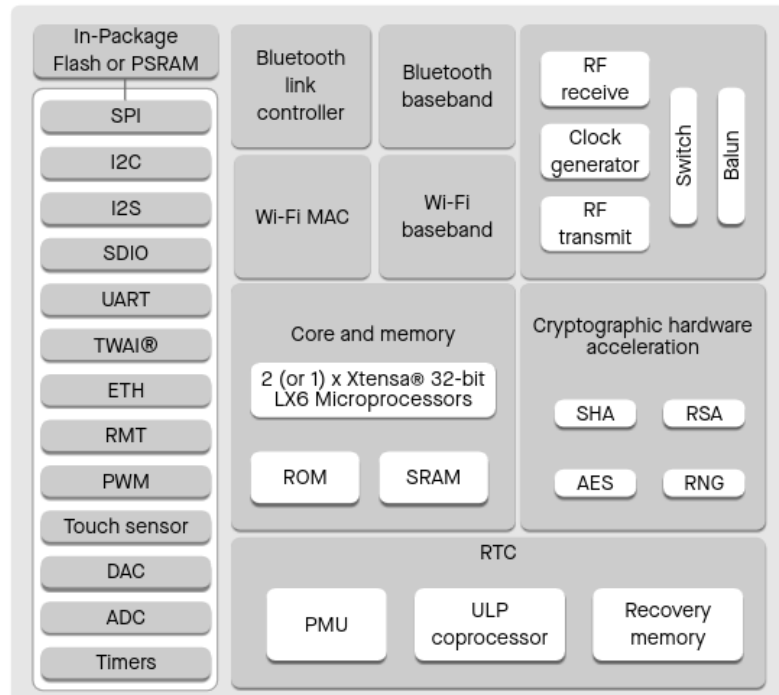


1. Arduino Uno Board

1. Digital pins-Use these pins with digitalRead(), digitalWrite(), and analogWrite(). analogWrite() works only on the pins with the PWM symbol.
2. Pin 13 LED-The only actuator built-in to your board. Besides being a handy target for your first blink sketch, this LED is very useful for debugging.
3. Power LED-Indicates that your Arduino is receiving power. Useful for debugging.
4. ATmegamicrocontroller.
5. Analog in-Use these pins with analogRead().
6. GND and 5V pins-Use these pins to provide +5V power and ground to your circuits.
7. Power connector-This is how you power your Arduino when it's not plugged into a USB port for power. Can accept voltages between 7-12V.
8. TX and RX LEDs-These LEDs indicate communication between your Arduino and your computer. Expect them to flicker rapidly during sketch upload as well as during serial communication. Useful for debugging.
9. USB port -Used for powering your Arduino Uno, uploading your sketches to your Arduino, and for communicating with your Arduino sketch (via Serial.println() etc.).
- 10.Reset button -Resets the ATmegamicrocontroller.



The ESP32, like Arduino, is a development board with a 32-bit microprocessor. ESP32 not only has built in WiFi but also has Bluetooth and Bluetooth Low Energy.



https://espressif-docs.readthedocs-hosted.com/projects/arduino-esp32/en/latest/tutorials/partition_table.html

The figure represents the internal architecture of an embedded microcontroller system, specifically the ESP32 SoC (System-on-Chip), commonly used for Internet of Things (IoT), automation, and wireless applications.

1. Core and Memory:

- **Microprocessors:** 2 (or 1) Xtensa® 32-bit LX6 microprocessors are the main processing cores.
- **ROM / SRAM:** Includes read-only memory (ROM) and static RAM for program/storage.
- **In-Package Flash or PSRAM:** Provides extra non-volatile or pseudo-static RAM memory.

2. Wireless Communications:

- **Wi-Fi Block:** Wi-Fi MAC and Wi-Fi baseband for handling 802.11 wireless networking.
- **Bluetooth Block:** Bluetooth link controller and baseband for Bluetooth communications.
- **RF Section:** Handles radio frequency transmission and reception with components such as RF receive/transmit, clock generator, switch, balun.

3. Peripherals:

- Connectivity and interface ports like SPI, I2C, I2S, SDIO, UART, TWAI® (CAN), Ethernet (ETH), PWM, RMT, DAC, ADC, timers, and a touch sensor.

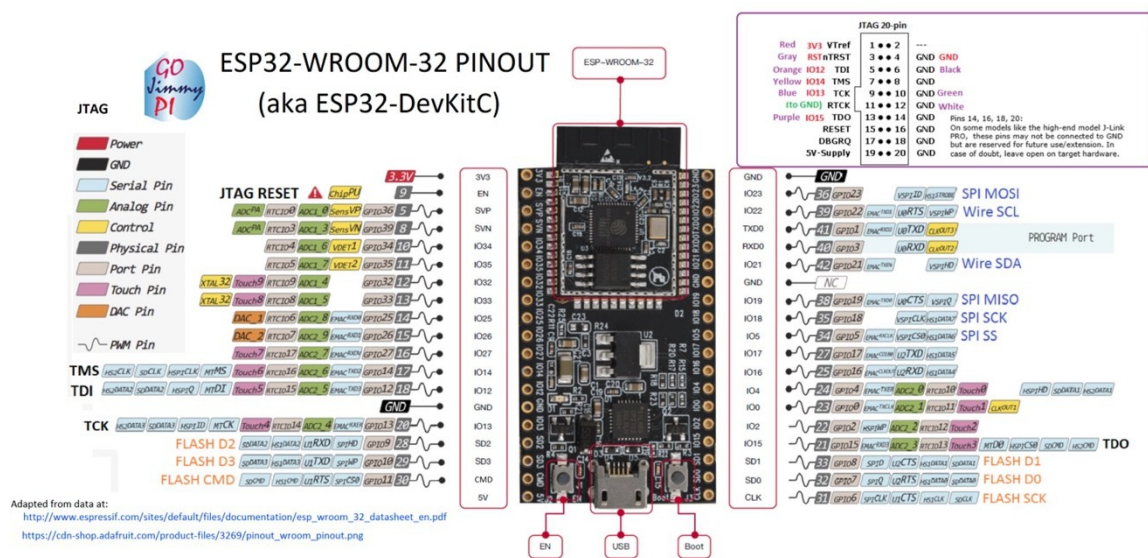
4. Cryptographic Hardware Acceleration:

- Dedicated blocks for accelerating encryption/hashing algorithms: SHA, RSA, AES, hardware random number generator (RNG).

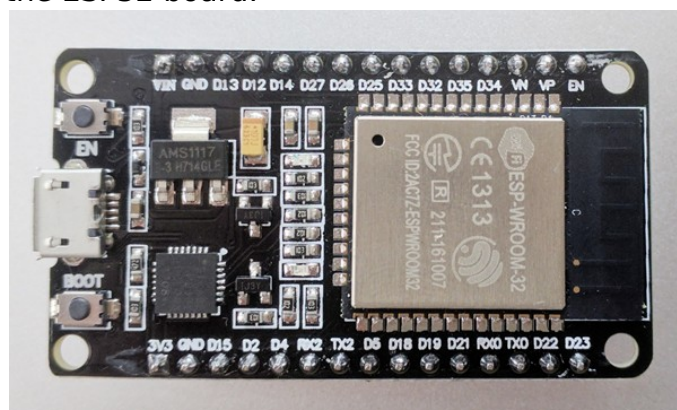
5. RTC (Real-Time Clock) Block:

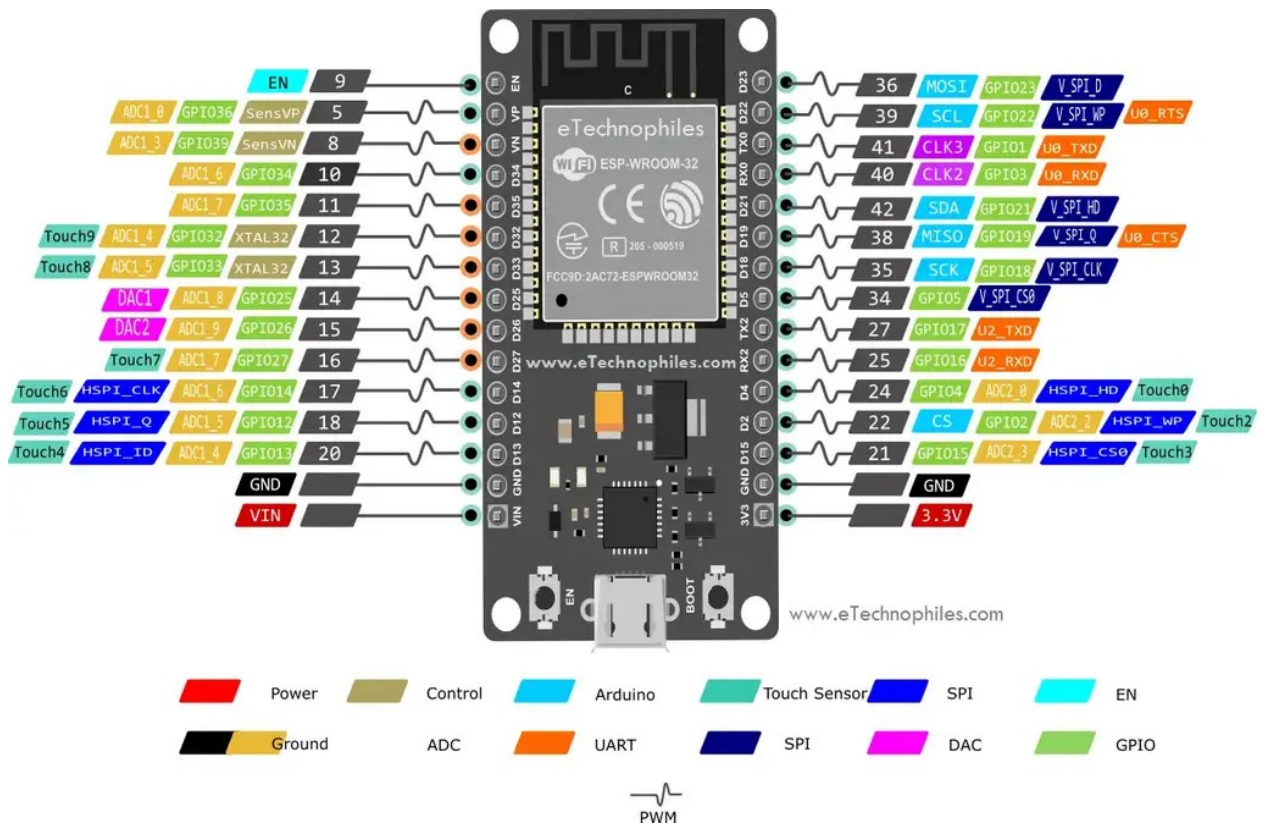
- **PMU:** Power Management Unit
- **ULP Coprocessor:** Ultra-Low-Power Coprocessor for running simple routines during deep sleep mode
- **Recovery Memory:** For storing data during low power or recovery operations

Examples of EPS32 boards:

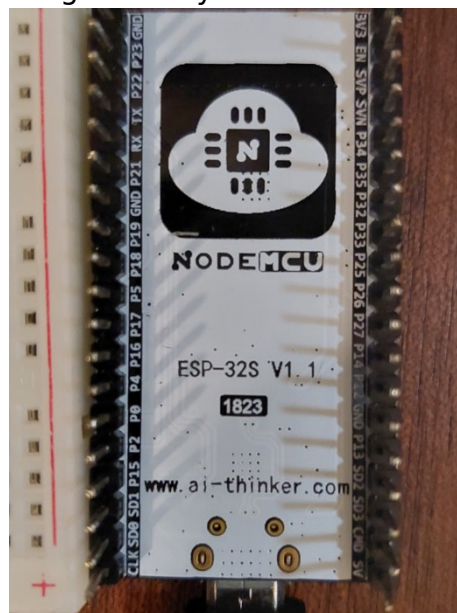


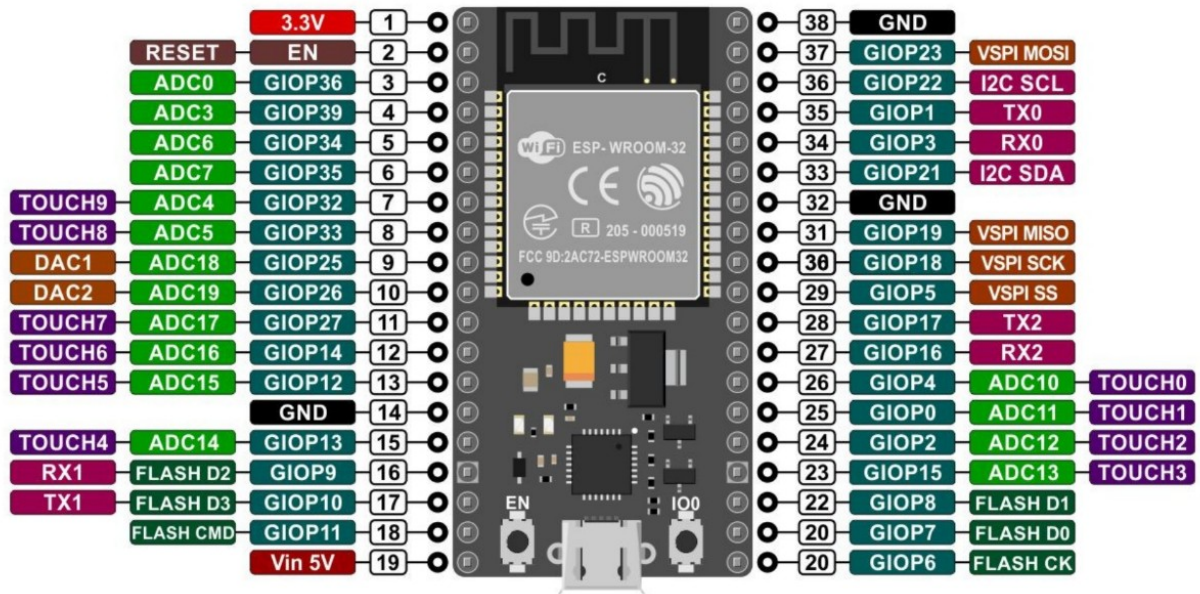
The pinout diagram of the ESP32 board:



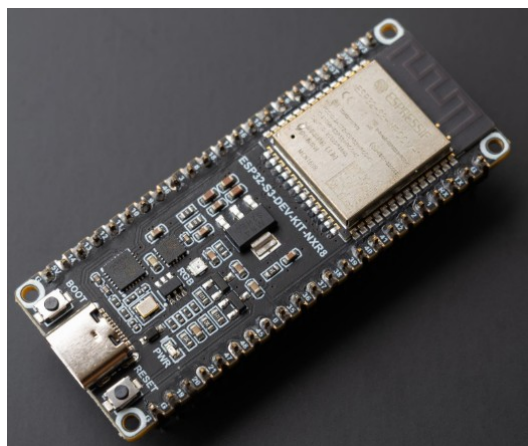
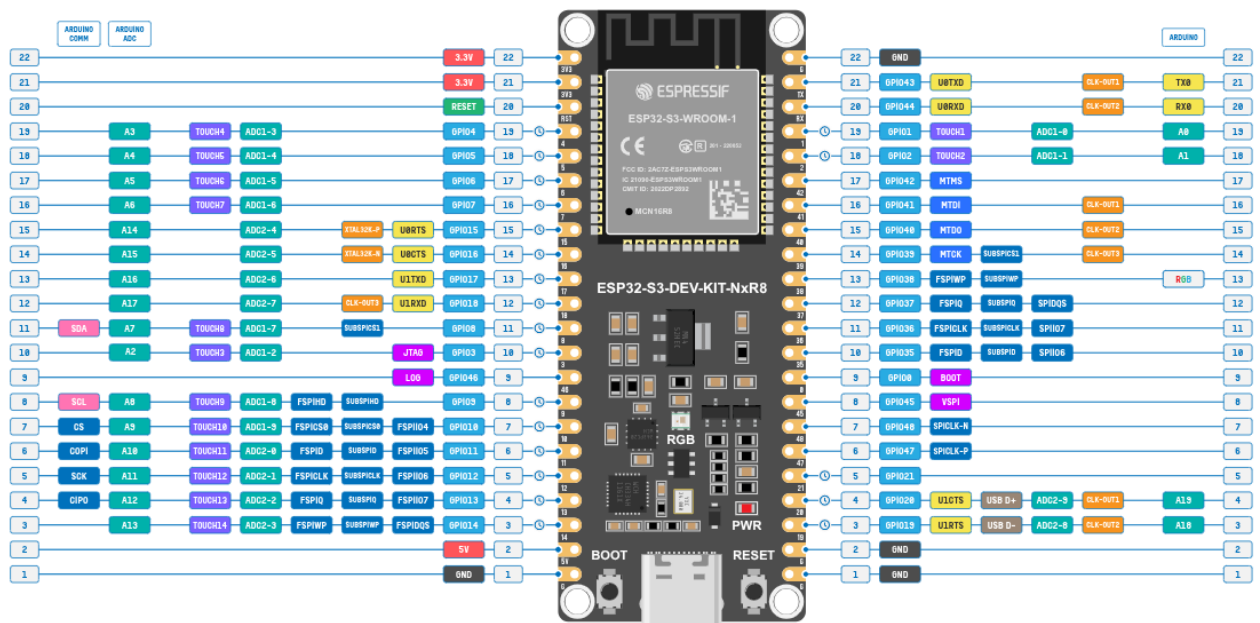


Verify what's the pinout diagram for your ESP32 board:





ESP32-S3 pinout:



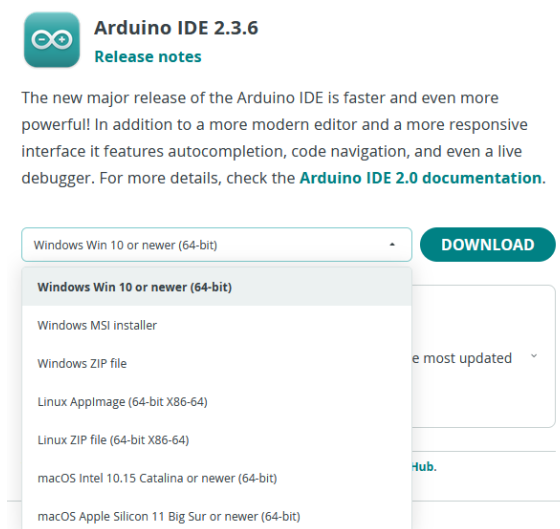
2. Arduino IDE

Before proceeding any further, verify that you have installed Arduino IDE with Arduino ESP32 board manager in your computer.

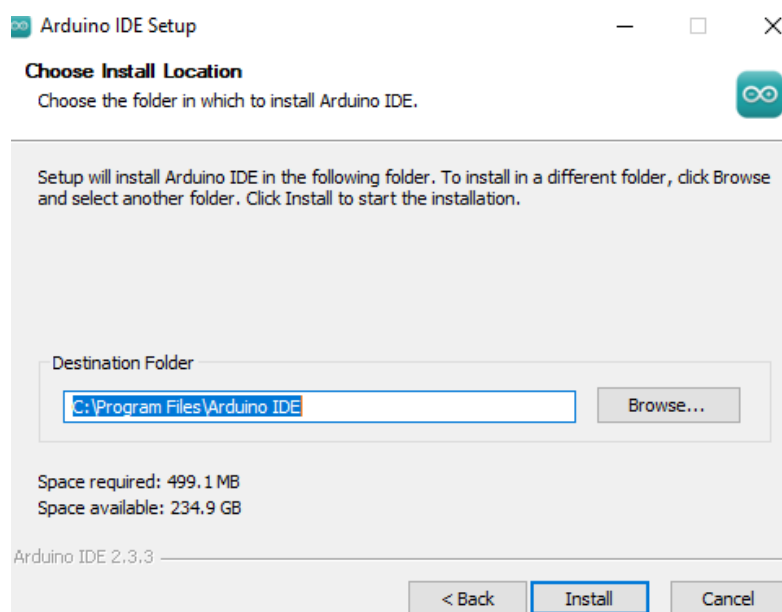
2.1 Installation

The Arduino IDE can be found at: <https://www.arduino.cc/en/software>

If not present, you can go through the following tutorial to install it.
<https://docs.espressif.com/projects/arduino-esp32/en/latest/installing.html>



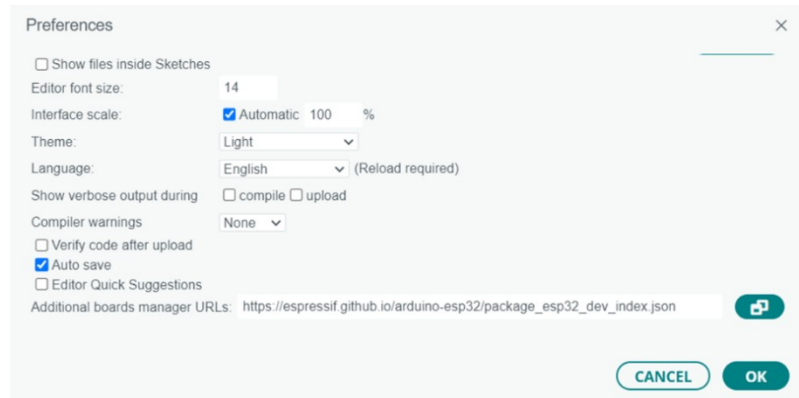
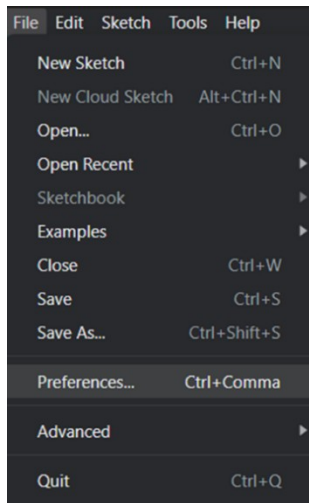
or download the zip file for your environment.



And add the:

https://espressif.github.io/arduino-esp32/package_esp32_dev_index.json

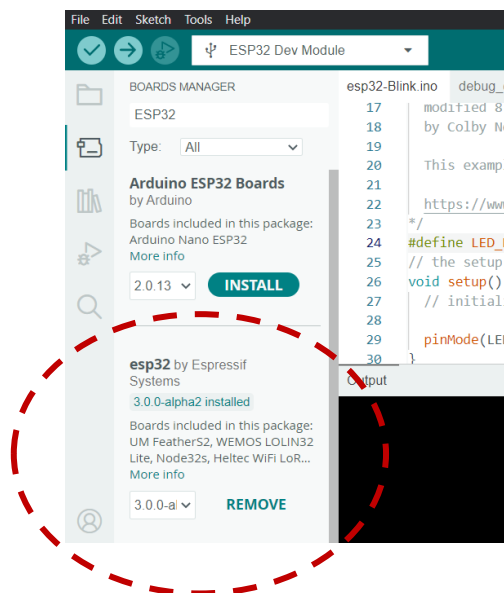
on the additional boards managers URLs.



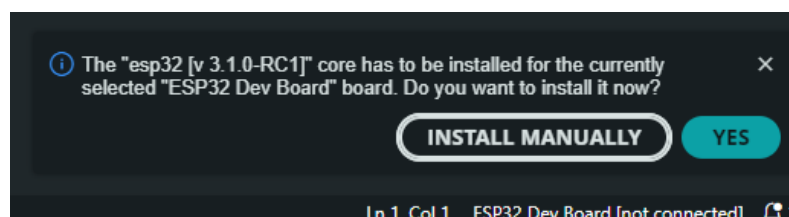
It's possible to have more than one type of equipment, use the "comma" or new line to add additional equipment's.

Open Boards Manager from Tools > Board menu and install esp32 platform (and do not forget to select your ESP32 board from Tools > Board menu after installation).

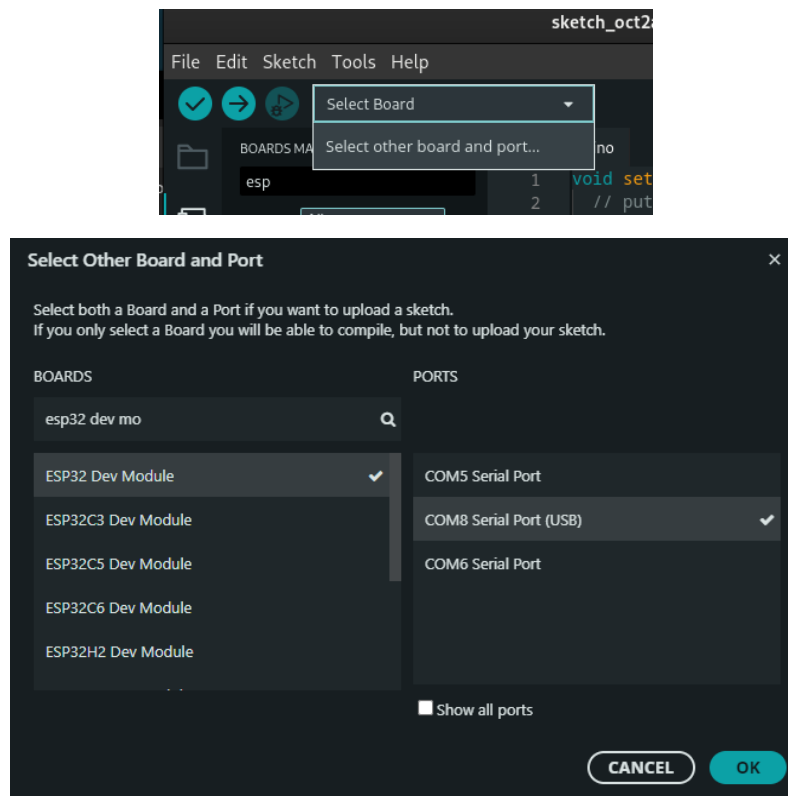
Select esp32 by Espressif Systems to install.



don't select RC or alpha (unstable versions), select a stable version.



this operation may take several minutes to finish.



or select ESP32S3 Dev Module, depending on your board.

2.2.1 Windows only

Windows 11 and arduino IDE 2.3 don't require the following procedures. Use only with legacy versions, or in case of problems.

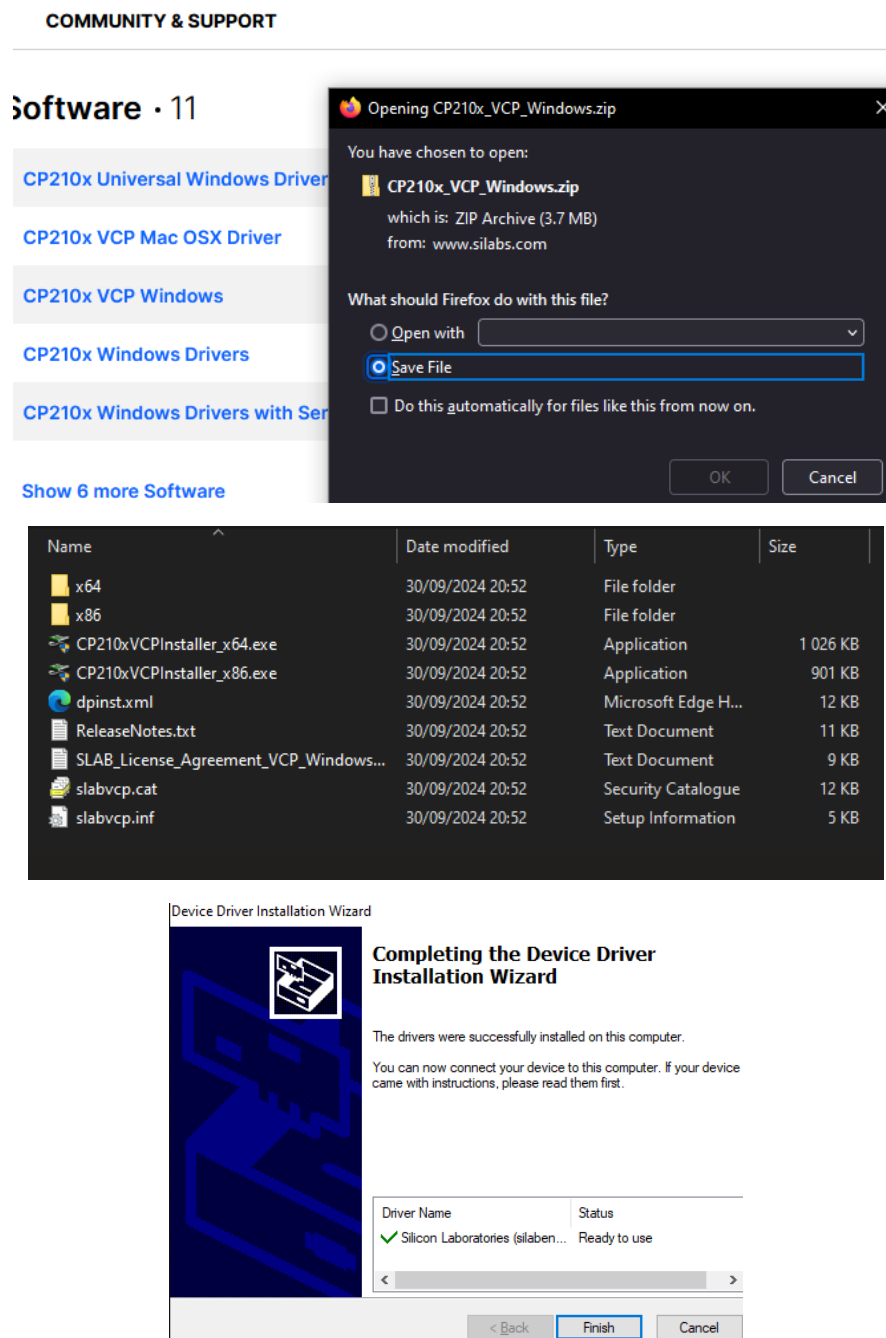
Verify if **"CP210x USB to UART Bridge"** is available at Device Manager.

If is not available:	If is available:
The screenshot shows the Windows Device Manager window. The 'Other devices' category is expanded, showing several 'Bluetooth Peripheral Device' entries. The 'Ports (COM & LPT)' category is not expanded, and the 'Silicon Labs CP210x USB to UART Bridge' is not visible in the list.	The screenshot shows the Windows Device Manager window. The 'Ports (COM & LPT)' category is expanded, and 'Silicon Labs CP210x USB to UART Bridge (COM3)' is listed and highlighted with a red rectangular box.

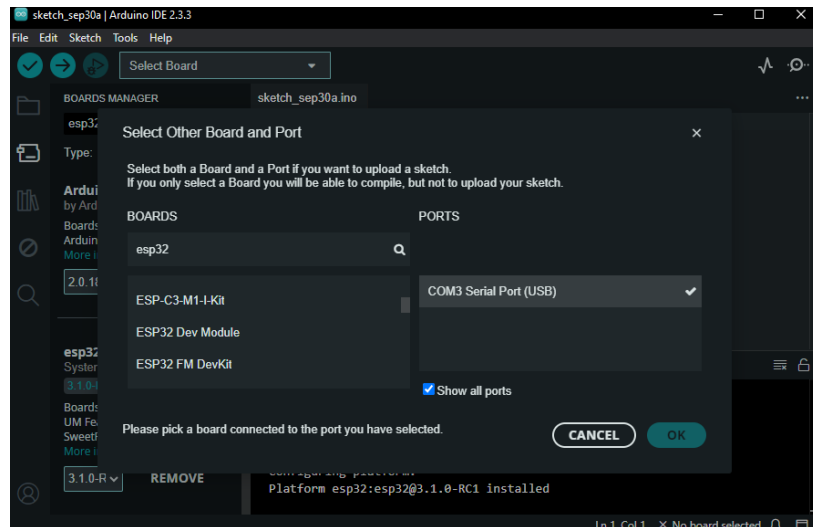
Follow the instructions if it's not available:

<https://docs.espressif.com/projects/esp-idf/en/latest/esp32/get-started/establish-serial-connection.html>

To install the driver: [CP210x USB to UART Bridge VCP Drivers](#)



and select “ESP32 Dev Module”:



2.2.2 Linux only useful commands

Debian versions requires only

```
sudo usermod -aG dialout "$USER"
```

but other linux flavors may need additional:

```
sudo chmod a+rw /dev/ttyusb0
```

```
sudo usermod -a -G tty $USER
```

```
python3 -m pip install pyserial
```

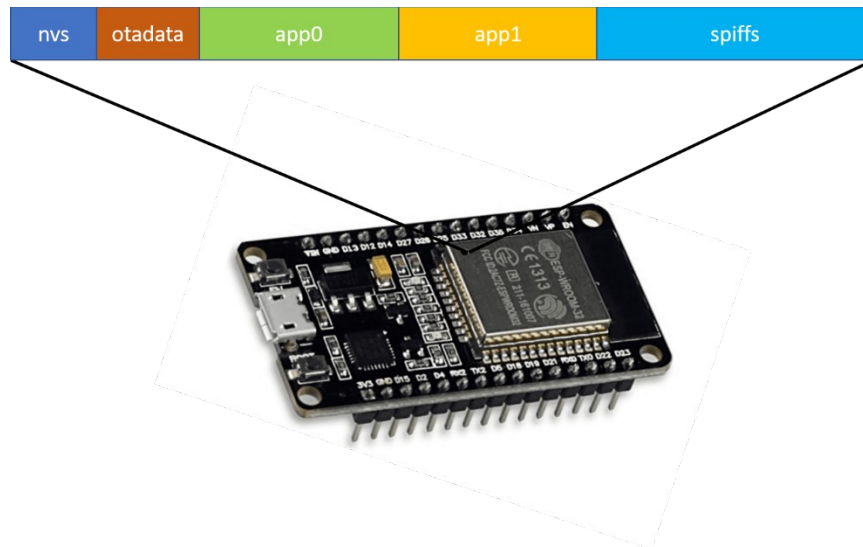
Or

```
sudo apt install python3-serial
```

2.2 Memory Management

The ESP32 Memory has the following characteristics:

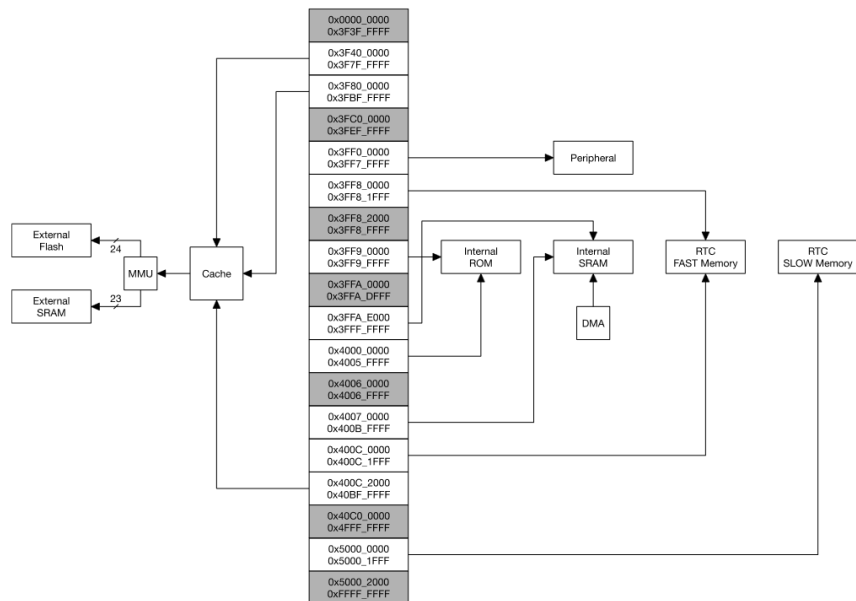
- The ESP32 has an external SPI Flash memory that's by default 4MB in size.
- Non-Volatile Storage: for user data storage
- 2x App Partitions: for user program storage + ability to perform OTA firmware updates
- OTAdata: used for OTA (Over-The-Air) firmware updates
- SPIFFS: SPI Flash File System for file storage and management



- The **ota** subtype is used to store the OTA information. The size of this partition should be a fixed size of 8kB (0x2000 bytes).
- The **nvs** partition subtype is used to define the partition to store general data, like the WiFi data, device PHY calibration data and any other data to be stored on the non-volatile memory. The recommended size for this partition is from 12kb to 64kb.
- The **FAT** partition subtype defines the FAT filesystem usage, and it is suitable for larger data and if this data is often updated and changed.
- The **spiffs** partition subtype defines the SPI flash filesystem usage, and it is also suitable for larger files and it also **performs the wear leveling and file system consistency check**.
- **APP The factory** partition subtype is the default application. The bootloader will set this partition as the default application initialization if no OTA partition is found, or the OTA partitions are empty.
- **Size** defines the amount of memory to be allocated on the partition. The size can be formatted as decimal, hex numbers (0x prefix), or using unit prefix K (kilo) or M (mega) i.e: 4096 = 4K = 0x1000.

The memory map (next figure) is crucial for ESP32 programming as it defines:

- Where code can be executed from
- Which memory regions are accessible during different power states
- How external memory expansion works
- Memory protection and access permissions



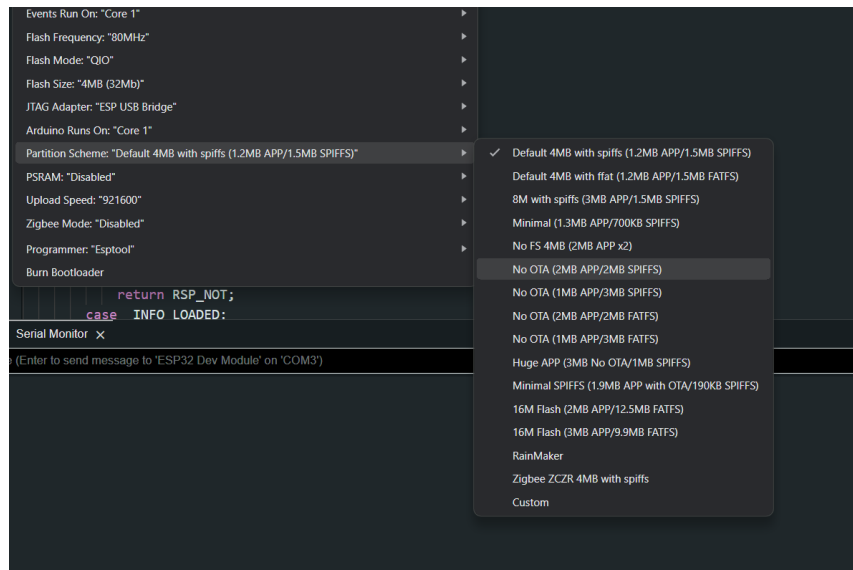
- 0x3F40_0000 0x3F7F_FFFF
 - 4 MB External Memory
- 0x3F80_0000 0x3FBF_FFFF
 - 4 MB External Memory

2.2.1 Arduino IDE Memory Configuration

Some users code may require extra memory than that comes by default (1.2MB for app and 1.5 for SPIFFS). The board has 4MB Flash, then we may try the no OTA (no on the air update) partitions.

If additional memory is required for App, go to Tools>Partitions Scheme and change the default to one of the following:

- No OTA (2MB ...
- Huge APP (3MB ...



This configuration solves memory errors such as “text section exceeds available space in board”:

```

Output  Serial Monitor
Sketch uses 1772380 bytes (135%) of program storage space. Maximum is 1310720 bytes.
Global variables use 61356 bytes (18%) of dynamic memory, leaving 266324 bytes for local variables. Maximum is 327680 bytes.
Sketch too big; see https://support.arduino.cc/hc/en-us/articles/360013825179 for tips on reducing it.
text section exceeds available space in board

Compilation error: text section exceeds available space in board

```

3. [Lab03.ex01] Internal LED blink

To visualize how Arduino programming functions copy the following code extract to Arduino IDE. Before uploading to the board do not forget to select the correct one (ESP 32 dev module and its port).

```

#define LED_BUILTIN 2
void setup() {
    // put your setup code here, to run once:
    delay(2000);
    // initialize digital pin LED_BUILTIN as an output.
    Serial.begin(9600);
    pinMode(LED_BUILTIN, OUTPUT);
}

void loop() {
    // put your main code here, to run repeatedly:
    digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the voltage level)
    delay(2000); // wait for 2 seconds
    digitalWrite(LED_BUILTIN, LOW); // turn the LED off by making the

```

```

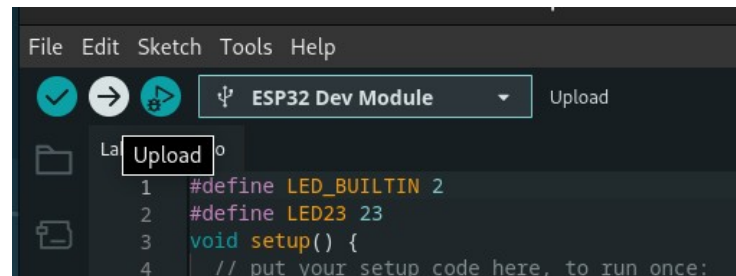
voltage LOW
    delay(1000);           // wait for a second
}

```

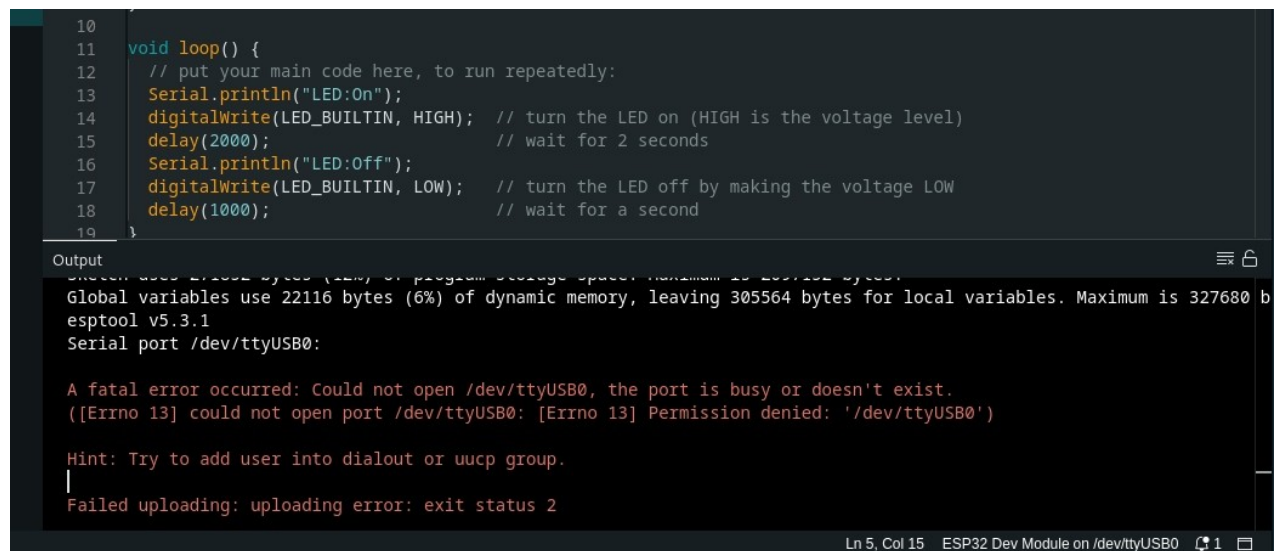
The code of Arduino is divided in two main functions: *setup* and *loop*. The *setup* function is first executed every time the Arduino is powered or every time a reset is performed. Next, the *loop* function will be executed in an infinite loop.

The only operation in the *setup* function is to define pin 2 as an output pin, meaning that we will be able to write on that pin. Pin 2 is internally connected to the LED on the Arduino board¹.

The loop function starts by writing 1 (HIGH or true) to pin 2, then it sleeps for 2 seconds, and pin goes to 0 (LOW or false), and finally it will sleep for another second. This will make the internal LED blink.



Upload, if error, check the configurations.



3.1 ESP32S3 [Lab03.ex01] Internal LED blink

ESP32 S3 doesn't have a built in led, but an addressable RGB/WS2812 ("NeoPixel") LED rather than a simple one-colour LED. Therefore, in Arduino IDE, go to Sketch → Include Library → Manage Libraries...

Search for and install Adafruit NeoPixel.

```

/*
  for esp32s3
  requires the installation
  1. In Arduino IDE, go to Sketch → Include Library → Manage Libraries...
  2. Search for and install Adafruit NeoPixel.
*/
#include <Adafruit_NeoPixel.h>
#define NEOPIXEL_PIN 38
#define NUM_PIXELS 1
Adafruit_NeoPixel pixel(NUM_PIXELS, NEOPIXEL_PIN, NEO_RGB + NEO_KHZ800);
int redValue, greenValue, blueValue;

void updatePixel() {
  pixel.setPixelColor(0, pixel.Color(redValue, greenValue, blueValue));
  pixel.show();
}

void setup() {
  // put your setup code here, to run once:
  delay(2000);
  // initialize digital pin as an output.
  Serial.begin(9600);
  redValue = 0;
  greenValue = 0;
  blueValue = 0;
  updatePixel();
}

void loop() {
  // put your main code here, to run repeatedly:

  redValue = 255;
  updatePixel();
  delay(1000);           // wait for a seconds

  redValue = 0;
  updatePixel();
  delay(1000);           // wait for a second
}

```

4. [Lab03.ex02] Hello World

When learning a new framework or language, it is common to greet the world. How can you greet the world without a display? Fortunately, Arduino IDE provides one.



Clicking in the Serial Monitor will open a display that shows any message printed by the Arduino.

As tradition it is time to say hello to the world. Copy the following code and upload it to the board.

```
void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600); // This value represents the Character encoder
  // (This value has to match the one present in the serial monitor)
}
void loop() {
  // put your main code here, to run repeatedly:
  Serial.println("Hello World");
  delay(1000); // wait for a second
}
```

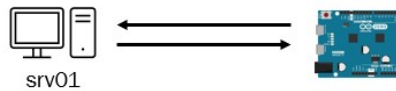
Using the serial monitor verify its output.

5. [Lab03.ex03] LED On

Change the Lab03.ex01 example to print in the Serial Monitor "LED:On" and "LED:Off" according to the digitalWrite.

6. [Lab03.ex04] Arduino Client – SRV01 Server – UDP connection

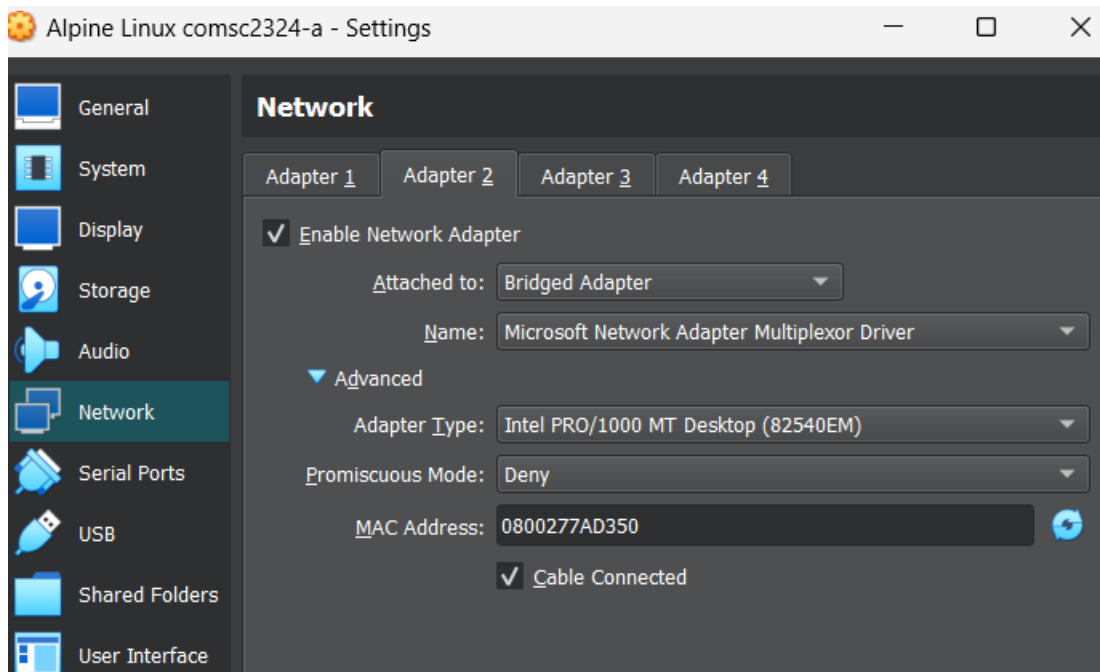
To implement this example and use the UDP protocol, we need to use the server code running on the SRV01 from PL1 class and the Arduino will act as a client.



Use the wireless smart router:

1. ssid: labs
2. Pass: 782edcwq#

And change the adapter 2 of SRV01 virtual machine to “**Bridge Adapter**” mode. Ex:



And use from the Lab01, the “2.2.2 UDP server (udp_srv.c) [Lab01.ex02]” at page 18 as an UDP server.

This following program connects to a WiFi network and uses UDP (User Datagram Protocol) to send a message ("hello world") to a server (srv01) at IP XXX.XXX.XXX.XXX, port 9999. It then waits for a response and prints it to the Serial Monitor. This repeats every 5 seconds.

Copy and upload the following UDP Client example in the ArduinoIDE:

```
#include <WiFi.h>
#include <WiFiUdp.h>
/* WiFi network name and password */
const char * ssid = "LABS";    // the SSID of the network
const char * pwd = "782edcwq#"; // the password of the network

// IP address to send UDP data to.
// it can be ip address of the server or
// a network broadcast address
// here is broadcast address
```

```

const char * udpAddress = "XXX.XXX.XXX.XXX"; //the address of the SRV01
const int udpPort = 9999;

//create UDP instance
WiFiUDP udp;
void setup(){
    delay(5000);
    Serial.begin(9600);
    //Connect to the WiFi network
    Serial.println("Connecting to the WiFi network...");
    WiFi.begin(ssid, pwd);
    // Wait for connection
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("");
    Serial.print("Connected to ");
    Serial.println(ssid);
    Serial.print("IP address: ");
    Serial.println(WiFi.localIP());
}

void loop(){
    //data will be sent to server
    uint8_t buffer[50] = "hello world";
    Serial.println("initializing udp and transfer buffer...");
    Serial.print("IP address: ");
    Serial.println(udpAddress);
    Serial.print("Port: ");
    Serial.println(udpPort);
    //This initializes udp and transfer buffer
    udp.beginPacket(udpAddress, udpPort);
    udp.write(buffer, 11);
    udp.endPacket();
    memset(buffer, 0, 50);
    //processing incoming packet, must be called before reading the buffer
    udp.parsePacket();
    //receive response from server, it will be ....
    if(udp.read(buffer, 50) > 0){
        Serial.print("Server to client: ");
        Serial.println((char *)buffer);
    }
    //Wait for 5 second
    delay(5000);
}

```


Program explained:

1. Import libraries

- `#include <WiFi.h>`: Provides functions for WiFi connection.
- `#include <WiFiUdp.h>`: Enables UDP networking.

2. WiFi Credentials

- `ssid` and `pwd`: Set the network's name and password. The code will try to connect using these.

3. UDP Setup

- `udpAddress` & `udpPort`: Specify the server's IP and port for UDP packets.
- `WiFiUDP udp;`: Creates a UDP object to handle communication.

4. `setup()` function

- Delays for 5 seconds at the start.
- Starts serial communication (`Serial.begin()`).
- Connects to WiFi using `WiFi.begin(ssid, pwd)`. Loops until connected, printing dots for progress.
- Once connected, prints the local IP assigned to the ESP32 (check the serial monitor).

5. `loop()` function

- Creates a 50-byte buffer and fills the start with the string "hello world".
- Prints status messages about sending UDP.
- Sends the buffer via UDP to the server (`beginPacket` → `write` → `endPacket`).
- Clears the buffer (`memset`) to prepare it for new incoming data.
- Looks for a received UDP packet (`udp.parsePacket()`).
- If a response is received (`udp.read() > 0`), prints the server's reply to Serial Monitor.
- Waits 5 seconds (`delay(5000)`) before repeating.

Questions:

Why is `memset()` called right after sending the message?

What could happen if the ESP32 is not connected to WiFi when `udp.beginPacket()` is called?

Why might using UDP be chosen over TCP in this use case?

Put here the answer:

7. [Lab03.ex05] Error handling

If the ESP32 is not connected to WiFi, the packet send operation (`udp.beginPacket`) will fail. No data will be transmitted, and subsequent `write` or `endPacket` will have no effect. The ESP32 won't be able to communicate over UDP, and there will be no response to read - usually, nothing appears on the receiving end.

Error handling is absent in the code, so it would just silently fail after printing status messages.

To add effective error handling to the code, you should:

- Check WiFi connection status before sending UDP packets.
- Check if UDP packet initialization and finishing (`beginPacket`&`endPacket`) succeeds.
- Add messages to alert the user if something fails.

Change the code accordingly.

8. [Lab03.ex06] Lights On

Change the **UDP server program** so that every 10 requests it **responds** once with "LightOn". Then, change the **Arduino client** to light the internal LED **if the message from server is "LightOn"**. Turn the LED off otherwise.

Useful function: `String.startsWith(...)`

<https://docs.arduino.cc/built-in-examples/strings/StringStartsWithEndsWith/>

9. [Lab03.ex07] Arduino TCP Server – Client connection

An Arduino is a blank page (limited by its storage capacity) that allows the programmer to implement different concepts. For example, and following our line of study, it is possible to create a **TCP server that listens and replies to client**.

This example is a basic echo server for the ESP32, using TCP sockets. It can be accessed through a browser or tools like telnet or netcat by connecting to the ESP32's IP address at port 8080. Any message sent to the server will be echoed back by the ESP32, which is useful for testing, debugging, or as a base for an HTTP server with more functionality.

Copy and upload the following TCP server example in the Arduino IDE:

```
// Load Wi-Fi library
#include <WiFi.h>
// Replace with your network credentials
const char* ssid = "<name_of_the_network>";
const char* password = "<password>";
// Set server port number to 8080
WiFiServer server(8080);
void setup() {
    Serial.begin(9600);
    // Connect to Wi-Fi network with SSID and password
    Serial.print("Connecting to ");
    Serial.println(ssid);
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    // Print local IP address and start web server
    Serial.println("");
    Serial.println("WiFi connected.");
    Serial.println("IP address: ");
    Serial.println(WiFi.localIP()); //prints the ip to which the server is
    listening
    server.begin();
}

void loop(){
    WiFiClient client = server.available();
    if (client) {
        while (client.connected()) {
            while (client.available() > 0) {
                String request = client.readString();
                Serial.print(request);
                sendResponse(&client, request);
            }
        }
        client.stop();
        Serial.println("Client disconnected");
    }
}
```

```
void sendResponse(WiFiClient* client, String response)
{
    client->print(response);
    // Debug print.
    Serial.println("sendResponse:");
    Serial.println(response);
}
```

How It Works

- **Setup:** Connects to WiFi and launches a TCP server on port 8080.
- **Loop:** Waits for incoming client connections; when data arrives, echoes it back to the client and prints it for debugging.
- **sendResponse:** Sends the data back to the client.

Using the TCP client implemented in the previous practical class (PL02), make the necessary changes to connect with the server running in the Arduino (**remember that both client and server have to be present in the same network**).

Use the wireless smart router:

1. ssid: labs
2. Pass: 782edcwq#

10. [Lab03.ex08] Arduino TCP Server – Light On

Using the example present in the previous question and the provided TCP server example, combine the code so that every time the **client writes "on"**, the Arduino **turns on the led**, and when the **client writes "off"** it **turns off the led**. The Server must reply to the client if it succeeded in turning on/off the led.